

Almacenamiento en el navegador

Cookies · localStorage · sessionStorage

Agenda

- Cookies (y su relación directa con HTTP)
- localStorage
- sessionStorage
- Cookies vs Web Storage

Cookies

Mecanismo de **gestión de estado en HTTP**, especificado en la **RFC 6265**, que permite emular sesiones sobre un protocolo que es, por diseño, **sin estado** (*stateless*).

- El servidor las establece mediante el header de respuesta `Set-Cookie`
- El agente de usuario las reenvía en cada petición con el header `Cookie`
- Se transmiten como parte del **tráfico HTTP**: no requieren JavaScript

Quién escribe	Cómo
Servidor	Header <code>Set-Cookie</code> en la respuesta HTTP
Cliente (JS)	<code>document.cookie = "..."</code> , solo si no es <code>HttpOnly</code>

```
HTTP/1.1 200 OK
Set-Cookie: sesion=abc123; Max-Age=3600; Secure; HttpOnly
```

```
GET /perfil HTTP/1.1
Cookie: sesion=abc123
```

Asociación de sesión en HTTP

HTTP es un protocolo **sin estado**: cada request se procesa de forma independiente. Para conservar la identidad del cliente, el servidor requiere un **identificador de sesión**.



Elemento	Ubicación	Contenido
Identificador de sesión	Cliente (cookie)	Token opaco (abc123)
Estado de sesión	Servidor	Usuario, permisos, expiración

La cookie transporta el identificador; el estado reside en el servidor.

Cookies · atributos importantes

Atributo	Para qué sirve
<code>Expires</code> / <code>Max-Age</code>	Cuándo expira (sin esto: cookie de sesión)
<code>Domain</code> / <code>Path</code>	En qué origen/ruta se envía
<code>Secure</code>	Solo se manda por HTTPS
<code>HttpOnly</code>	JS no puede leerla (<code>document.cookie</code>)
<code>SameSite</code>	Controla envío en requests cross-site (CSRF)

Cookies · desde JS

```
// Escribir
document.cookie = "tema=oscuro; path=/; max-age=3600"

// Leer (solo las que NO son HttpOnly)
console.log(document.cookie)
// "tema=oscuro; otraCookie=valor"
```

- API vieja e incómoda: todo es un solo string, hay que parsearlo a mano
- Límite chico: **~4 KB por cookie**
- Se mandan en **cada request** al servidor → pesan en el tráfico

Cookies · casos de uso

Caso de uso	Ejemplo
Sesión de usuario / autenticación	<code>sesion=abc123; HttpOnly</code> tras el login
Preferencia leída por el servidor	<code>idioma=es-AR</code> para renderizar la página (SSR)
Tracking de terceros	<code>_ga=GA1.2...</code> (Google Analytics), <code>fbp=...</code> (Meta Pixel)

Web Storage API

`localStorage` y `sessionStorage` comparten la misma interfaz: `Storage`

```
storage.setItem(clave, valor)
storage.getItem(clave)
storage.removeItem(clave)
storage.clear()
```

- A diferencia de las cookies, **nunca viajan** en los requests HTTP
 - Solo existen del lado del cliente, accesibles por JS
 - Pares clave/valor, siempre **strings**
 - Aislados **por origen** (protocolo + dominio + puerto)
- Quién escribe:** únicamente JavaScript ejecutándose en el cliente. El servidor no tiene forma de escribir directamente: no hay un header equivalente a `Set-Cookie`.

localStorage

Almacenamiento **persistente**, por origen.

- Sin fecha de expiración
- Sobrevive a cerrar la pestaña, el navegador o reiniciar el equipo
- Se borra solo explícitamente (`clear()` , `removeItem()`)
- Límite típico: **~5–10 MB** por origen
- Compartido entre **todas las pestañas** del mismo origen

```
localStorage.setItem("tema", "oscuro")
const tema = localStorage.getItem("tema")
localStorage.removeItem("tema")
```

localStorage · casos de uso

Caso de uso	Ejemplo
Preferencia de UI	Tema oscuro/claro, idioma elegido en el cliente
Carrito de compras	Ítems agregados, persisten entre visitas
Cache de datos de API	Evitar refetch de datos que cambian poco
Borrador de formulario	Autosave para no perder texto largo si se cierra la pestaña
Feature flags / A-B testing	Bandera asignada al cliente, sin ida y vuelta al servidor

sessionStorage

Almacenamiento **temporal**, acotado a una pestaña.

- Vive mientras la **pestaña** esté abierta
- Se borra al **cerrarla** (no al recargar la página)
- **No** se comparte entre pestañas, ni del mismo origen
- Mismo límite de tamaño que `localStorage`

```
sessionStorage.setItem("pasoActual", "2")  
const paso = sessionStorage.getItem("pasoActual")
```

sessionStorage · casos de uso

Caso de uso	Ejemplo
Formulario / wizard multi-paso	Paso actual y datos ingresados en esa pestaña
Checkout en curso	Datos de compra que no deben sobrevivir a la pestaña
state / nonce de OAuth	Valor temporal validado al volver del redirect
Filtros de una tabla	Orden/búsqueda propios de esa pestaña, no de otras

Cookies vs Web Storage

	Cookies	localStorage	sessionStorage
Viaja en cada request HTTP	Sí	No	No
Quién escribe	Servidor o JS	Solo JS	Solo JS
Quién lee	Servidor y JS	Solo JS	Solo JS
Duración	Configurable	Persistente	Por pestaña
Tamaño	~4 KB	~5–10 MB	~5–10 MB
Uso típico	Sesión, auth	Preferencias, cache	Estado de un flujo

Buenas prácticas de seguridad

- Datos sensibles (tokens, sesión) → **cookie** **HttpOnly** + **Secure** , no **localStorage**
- **localStorage** / **sessionStorage** son legibles por **cualquier script** → vulnerables a XSS
- No guardar más de lo necesario, y siempre validar/limitar tamaño

Resumen

- **Cookies:** viajan en cada request HTTP, ideales para sesión/auth
- **localStorage:** persistente, solo cliente, todas las pestañas
- **sessionStorage:** temporal, solo cliente, una pestaña

Speaker notes